

Projeto_LAMD - Point Dog

Documentação do Projeto — Sistema de Agendamento de Banho e Tosa

1. Descrição do Domínio

O sistema proposto é uma ****plataforma de agendamento de banho e tosa para cães****, que conecta donos de pets a prestadores de serviço (pet shops) de forma digital e assíncrona.

O problema central é a ineficiência do processo atual: agendamentos feitos por telefone ou WhatsApp, sem histórico centralizado, sem confirmação automática e sem rastreamento de status em tempo real. O sistema resolve isso fornecendo uma interface mobile para cada perfil de usuário e uma comunicação orientada a eventos entre os componentes.

2. Justificativa

A escolha do domínio se justifica pela:

- **Relevância prática:** mercado pet em crescimento constante no Brasil, com alta demanda por digitalização de serviços locais.
- **Adequação técnica:** o fluxo de agendamento — solicitação, aceite, execução, conclusão — é naturalmente orientado a eventos (EDA), alinhando domínio e arquitetura.
- **Complexidade controlada:** entidades bem delimitadas (usuário, pet, serviço, agendamento) permitem demonstrar Clean Architecture sem over-engineering.

3. Perfis de Usuário

3.1 Cliente (Dono do Pet)

Usuário final que utiliza o app para agendar serviços para seu animal. Suas responsabilidades no sistema:

- Cadastrar seus pets (nome, raça, porte, observações)
- Visualizar serviços disponíveis e seus preços
- Criar e acompanhar agendamentos
- Receber notificações sobre aceite, recusa ou conclusão do serviço

3.2 Prestador de Serviço

Profissional ou estabelecimento que oferece os serviços de banho e/ou tosa. Suas responsabilidades:

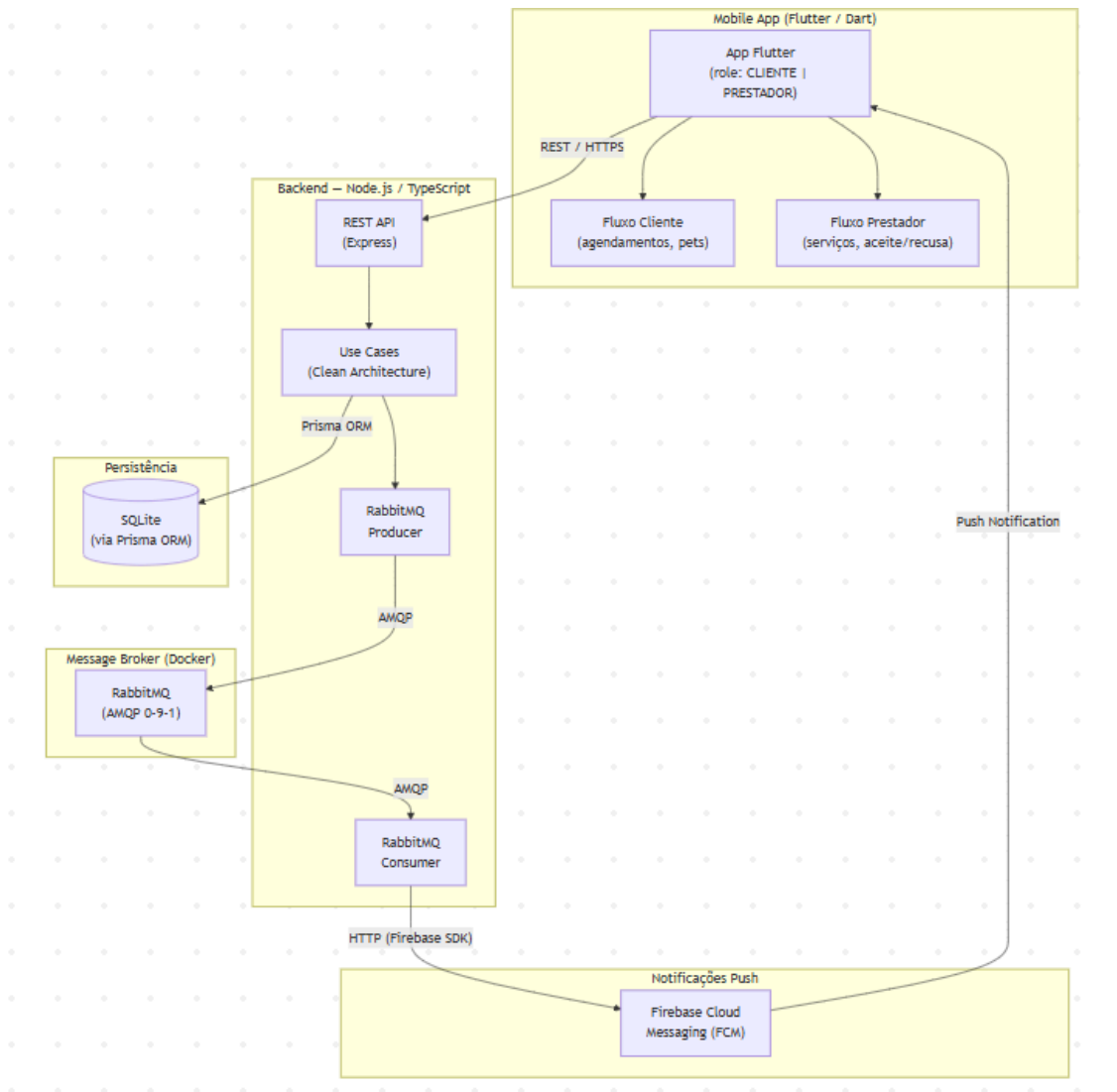
- Cadastrar os serviços que oferece (tipo, preço, duração estimada)
- Receber notificações de novas solicitações em tempo real
- Aceitar ou recusar agendamentos
- Atualizar o status do serviço durante a execução

4. Principais Funcionalidades

Funcionalidade	Perfil	Sprint
Cadastro e autenticação (JWT)	Ambos ▾	1 ▾
CRUD de pets	Cliente ▾	1 ▾
CRUD de serviços	Prestador ▾	1 ▾
Criação e listagem de agendamentos	Cliente ▾	1 ▾
Atualização de status do agendamento	Prestador ▾	1 ▾
Notificação assíncrona via RabbitMQ	Ambos ▾	2 ▾
App mobile Flutter — fluxo Cliente	Cliente ▾	3 ▾
App mobile Flutter — fluxo Prestador	Prestador ▾	4 ▾
Notificações push via Firebase	Ambos ▾	4 ▾

Visão Geral do Sistema

O sistema é composto por um aplicativo mobile Flutter com navegação baseada em roles (CLIENTE e PRESTADOR), um backend REST (Node.js/TypeScript), um message broker (RabbitMQ) e um serviço de notificações push (Firebase). A comunicação entre componentes utiliza dois protocolos distintos: REST/HTTPS para operações síncronas e AMQP para eventos assíncronos.



Protocolos de Comunicação

Conexão	Protocolo	Justificativa
App ↔ Backend	REST / HTTPS ▾	Operações síncronas (CRUD, auth)
Backend → RabbitMQ	AMQP ▾	Publicação de eventos de domínio
RabbitMQ → Backend	AMQP ▾	Consumo assíncrono de eventos

Backend → Firebase	HTTP (SDK) ▾	Disparo de push notifications
Firebase → Apps	FCM ▾	Notificações push nativas iOS/Android

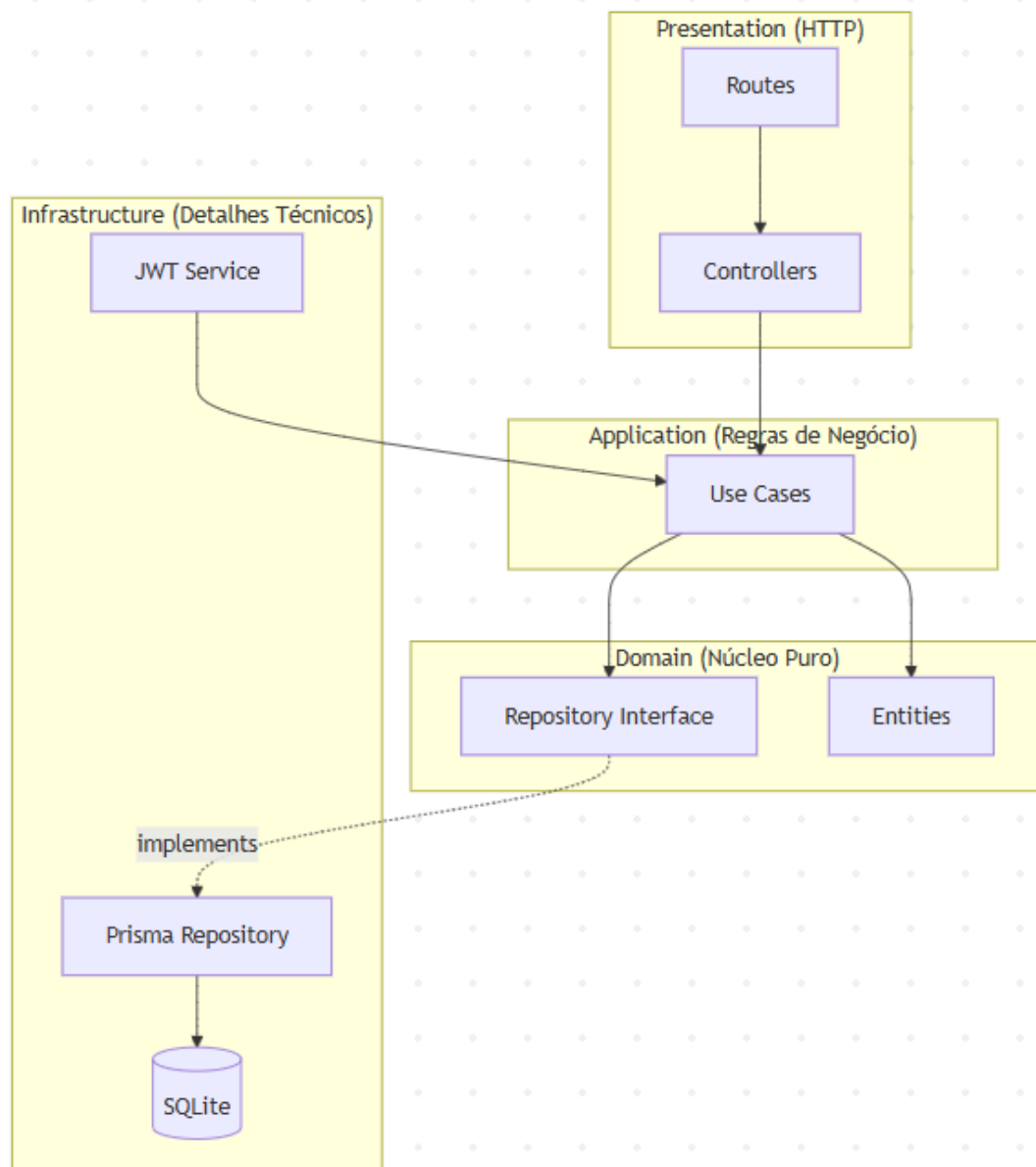
Componentes por Sprint

Componente	Sprint de Introdução
REST API + Clean Architecture + SQLite	Sprint 1
RabbitMQ (Docker) + Producers/Consumers	Sprint 2
App Flutter (fluxo Cliente)	Sprint 3
App Flutter (fluxo Prestador) + Firebase FCM	Sprint 4

Referências de estrutura

Diagrama de Camadas — Clean ArchitectureRegra de Dependência

A dependência sempre flui **de fora para dentro**. Camadas externas conhecem as internas; camadas internas nunca conhecem as externas.



- **Presentation** → chama Use Cases via Controllers
- **Application** → orquestra regras de negócio, depende apenas de interfaces
- **Domain** → entidades e contratos puros, zero dependências externas
- **Infrastructure** → implementações concretas (Prisma, JWT), injetadas via factory

Estrutura de Pastas

```
backend/
├── src/
│   ├── modules/
│   │   ├── users/
│   │   │   ├── domain/
│   │   │   │   ├── user.entity.ts
│   │   │   │   └── user.repository.ts
│   │   │   ├── application/
│   │   │   │   └── use-cases/
│   │   │   │       ├── register-user.usecase.ts
│   │   │   │       └── login-user.usecase.ts
│   │   │   ├── infrastructure/
│   │   │   │   └── prisma-user.repository.ts
│   │   │   ├── presentation/
│   │   │   │   ├── user.controller.ts
│   │   │   │   └── user.routes.ts
│   │   │   └── index.ts
│   │   ├── pets/
│   │   │   └── ... (mesma estrutura)
│   │   ├── services/
│   │   │   └── ... (mesma estrutura)
│   │   └── appointments/
│   │       └── ... (mesma estrutura)
│   ├── shared/
│   │   ├── middlewares/
│   │   │   └── auth.middleware.ts
│   │   ├── errors/
│   │   │   └── app-error.ts
│   │   └── container/
│   │       └── index.ts
│   ├── app.ts
│   └── server.ts
├── prisma/
│   └── schema.prisma
├── docker-compose.yml
├── Dockerfile
└── package.json
```